DESIGN AND ANALYSIS OF ALGORITHMS (CC342)

CC342 — Week 1: Lectures 1–2

Chapter 1: Introduction to Algorithmic Problems

1.1 What is an Algorithm?

An **algorithm** is a finite sequence of clear instructions for solving a problem. In simple words: An algorithm tells us **what steps to perform, in what order, to obtain the required answer**. For example, suppose we want to find the largest number in a list:

```
</>

5, 8, 2, 10, 4
```

A simple algorithm could be:

```
< >

1. Assume the first number is the largest.
2. Compare it with the next number.
3. If the next number is larger, update the largest value.
4. Continue until all numbers have been checked.
5. Return the largest value.
```

The answer is:

```
</>

10
```

An algorithm is therefore different from a **program**. An algorithm describes the logical solution, while a program implements that algorithm in a programming language such as Python, C++, Java, or C.

1.2 What is Algorithm Analysis?

Two different algorithms may solve the same problem but require very different amounts of time. Therefore, in **Design and Analysis of Algorithms**, we normally ask two major questions:

Design question

How can we solve the problem?

Analysis question

How efficient is our solution?

Efficiency usually includes:

- **Time efficiency** — how long the algorithm takes;
- **Space efficiency** — how much memory the algorithm requires.

The central lesson is:

> Choosing the correct algorithm can make an enormous difference.

This becomes particularly clear when we compare two different algorithms for computing Fibonacci numbers later in this chapter.

1.3 Types of Problems

These problems can be divided into three broad categories:

1. straightforward/simple programming problems;
2. algorithmic problems;
3. artificial intelligence problems.

The course primarily focuses on problems that are clearly formulated but difficult to solve efficiently.

1.4 Straightforward Programming Problems

A **straightforward programming problem** is one for which the natural solution is already reasonably efficient. These problems have a straightforward implementation and an already efficient natural solution. Examples discussed in lectures:

Example 1: Display Given Text.

Suppose the computer receives:

```
</>

Design and Analysis of Algorithms
```

The task is to display the same text. No complicated algorithmic decision is involved.

Example 2: Copy a File.

Suppose we want to copy:

```
</>

source.txt
```

to:

```
</>

backup.txt
```

Conceptually, the computer:

```
</>

reads data from source.txt
writes the data to backup.txt
```

Normally, every piece of information must be read and written. No magical algorithm avoids processing the data itself.

1.5 Simple Programming Problems

Another example is searching for a word in a piece of text.

Suppose we have:

```
</>

The design and analysis of algorithms is important.
```

and want to find:

```
</>

analysis
```

One natural technique is a **linear scan**. Simple programming problems are ones where a linear scan may be necessary and where the obvious approach already works reasonably well.

1.6 Linear Scan

A **linear scan**, also called **linear search**, means examining elements one after another.

Suppose:

```
</>

A = [13, 7, 21, 5, 18]
```

and we want to find:

```
</>

5
```

The search proceeds as follows:

Position	Value	Is it 5?
0	13	No
1	7	No
2	21	No
3	5	Yes

The algorithm stops after finding 5. A basic version is:

```
</>

LinearSearch(A, target)

for each element x in A:
    if x = target:
        return FOUND

return NOT FOUND
```

If there are n elements, the algorithm may have to examine all n elements.

Therefore, its running time is:

$$O(n)$$

The notation $O(n)$ will be studied more formally later. For now, it simply means that as the number of elements grows, the amount of work grows approximately in direct proportion to the number of elements.

1.7 Algorithmic Problems

Algorithmic problems are more interesting.

They are described as problems where:

- it may not initially be clear what to do;
- simple ideas may be too slow;
- there is room for optimization.

This is where **algorithm design** becomes important.

1.8 Example: Shortest Path Problem

An example: Find the shortest path between locations.

Imagine the following road network:

```
</>

A ----4---- B
|           |
2           3
|           |
C ----1---- D
```

The numbers could represent distances.

Suppose we want to travel from A to D.

Possible paths include:

```
</>

A → B → D
distance = 4 + 3 = 7
```

and

```
</>

A → C → D
distance = 2 + 1 = 3
```

Therefore:

```
A → C → D
```

is the shorter route.

Important terminology

A road-network problem is often represented using a **graph**.

A graph contains:

Vertex or node:

An object or location.

For example:

```
Plain text
A, B, C, D
```

Edge:

A connection between two vertices.

Weight:

A numerical value associated with an edge, such as:

- distance;
- time;
- cost;
- fuel

As additional background, common shortest-path algorithms include **Breadth-First Search** for some unweighted graphs and **Dijkstra's algorithm** for many non-negative weighted graphs.

1.9 Example: Assigning Students to Dorm Rooms

Another question to be asked here:

Find the best assignment of students to dorm rooms.

Suppose:

```
Student A prefers Room 2
Student B prefers Room 1
Student C prefers Room 3
```

The problem is not simply assigning any room. We may want an assignment that maximizes overall satisfaction. This belongs to a family of problems known as **assignment problems**. An assignment problem normally contains:

- a set of objects;
- another set of resources;

- preferences or costs;
- goals such as minimizing cost or
- Additional information

These problems can often be represented using:

- bipartite graphs;
- matching;
- optimization techniques.

The **Hungarian algorithm** is one well-known algorithm for the classical assignment problem, although it is not introduced in Week 1.

1.10 Example: Measuring Similarity Between Documents

The next problem is:

> Measure the similarity of documents.

Suppose we have:

Document A

```
algorithms improve computer performance
```

Document B

```

efficient algorithms improve performance
```

These documents contain several related words.

A computer may calculate a numerical **similarity score**.

For example:

```
0      → completely different
1      → extremely similar
```

Common ways of comparing documents include:

- word overlap;
- Jaccard similarity;
- cosine similarity;
- TF-IDF vectors;
- Modern neural embeddings

The important idea for this course is that it is not always immediately obvious which computational procedure gives the best answer efficiently.

1.11 Optimization

Optimization means improving a solution according to some objective.

For example:

```
</>

Find the shortest route.
```

The objective is:

```
</>

minimize distance
```

In student-room assignment:

```
</>

maximize satisfaction
```

In scheduling:

```
</>

minimize completion time
```

Optimization does **not** simply mean making code shorter. It normally means finding the best or a better solution according to a measurable criterion.

1.12 Artificial Intelligence Problems

The first two weeks' lectures distinguish some AI problems because even clearly defining the desired result can sometimes be difficult.

Examples discussed in lectures include:

- understanding natural language;
- identifying objects in photographs;
- playing games well.

These examples go beyond the primary scope of this introductory algorithms course.

Natural Language Understanding

Consider:

```
</>

I saw the man with the telescope.
```

This sentence can mean:

1. I used a telescope to see the man.

or:

2. I saw a man who was carrying a telescope.

Humans use context to understand language. For a machine, this can be difficult.

Object Recognition

Given an image, a computer may need to answer:

```
</>

Is this a cat?
Is this a car?
Where is the person?
```

This requires interpretation rather than simple arithmetic.

Game Playing

In chess, for example, the computer must decide:

```
</>

Which move should I make?
```

There may be an enormous number of possible future move sequences. Modern AI still relies heavily on algorithms, but these lectures separate such problems because their formulation and evaluation can be more complex than the classical algorithmic problems being introduced.

Chapter 2: Fibonacci Numbers

The next major topic is the **Fibonacci sequence**. The learning objective is not merely to calculate Fibonacci numbers. The real goal is to demonstrate how a seemingly reasonable algorithm can be extremely slow, while a better-designed algorithm can solve the same problem.

2.1 Definition of Fibonacci Numbers

The Fibonacci sequence is defined as:

$$F_n = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F_{n-1} + F_{n-2} & n > 1 \end{cases}$$

Therefore generate:

$$0,1,1,2,3,5,8,13,21,34,\ldots$$

2.2 Understanding the Notation

$$F_n$$

Means:

The Fibonacci number at position n.

For example:

$$F_0 = 0$$

$$F_1 = 1$$

$$F_2 = 1$$

$$F_3 = 2$$

F_{n-1}

Means:

the previous Fibonacci number.

F_{n-2}

Means:

Fibonacci number two positions earlier.

2.3 Calculating Fibonacci Numbers Manually

Starting with:

$$F_0 = 0$$

and

$$F_1 = 1$$

we obtain:

$$F_2 = F_1 + F_0$$

$$= 1 + 0 = 1$$

Then:

$$F_3 = F_2 + F_1$$

$$= 1 + 1 = 2$$

Then:

$$F_4 = F_3 + F_2$$

$$= 2 + 1 = 3$$

Then:

$$F_5 = F_4 + F_3$$

$$= 3 + 2 = 5$$

Then:

$$F_6 = F_5 + F_4$$

$$= 5 + 3 = 8$$

So:

```
F0 = 0
F1 = 1
F2 = 1
F3 = 2
F4 = 3
F5 = 5
F6 = 8
```

2.4 Recurrence Relation

The expression

$$F_n = F_{n-1} + F_{n-2}$$

is called a **recurrence relation**.

A recurrence relation defines a value using previous values of the same sequence.

For example:

$$F_8$$

depends on:

$$F_7$$

and:

$$F_6$$

This recursive mathematical definition naturally suggests a recursive computer algorithm. As we will see, however, a mathematical definition that looks elegant does not automatically produce an efficient program.

2.5 Base Cases

The values

$$F_0 = 0$$

and

$$F_1 = 1$$

are called **base cases**.

A base case is a situation whose answer is known directly. Base cases are extremely important in recursion because they stop the recursive calls. Without an appropriate base case, recursion may continue indefinitely.

2.6 Historical Motivation: Rabbit Population

The Fibonacci numbers were developed in connection with a rabbit population problem. The classical simplified model assumes things such as:

- rabbit pairs mature;
- mature pairs reproduce;
- rabbits do not die during the experiment;
- reproduction happens at regular intervals.

Under these idealized assumptions, the number of rabbit pairs follows a Fibonacci-like pattern. The important point is not biological realism. It is that a recurrence relation can model how one state develops from previous states.

2.7 Fibonacci Numbers Grow Rapidly

The following lemma holds for Fibonacci:

$$F \geq 2^{n/2}$$

for:

$$n \geq 6$$

This inequality tells us that Fibonacci numbers grow at least exponentially fast. A lemma is not simply a guess. It is a proven result.

You may think of:

```
</>

Theorem → major mathematical result
Lemma   → supporting proven result
```

The distinction is mostly about purpose and importance, not mathematical validity.

2.9 Proof by Induction

The notes prove the rapid-growth result using **mathematical induction**. Induction normally contains:

1. a base case;
2. an induction hypothesis;
3. an inductive step.

Base Cases

For $n = 6$:

$$F_6 = 8$$

and:

$$2^{6/2} = 2^3 = 8$$

Therefore:

$$F_6 \geq 2^{6/2} \text{ is true.}$$

For $n = 7$:

$$F_7 = 13$$

while:

$$2^{7/2} \approx 11.31$$

Therefore:

$$13 \geq 11.31 \text{ is also true.}$$

2.10 Induction Hypothesis

Assume that the result is true for the previously required Fibonacci numbers:

$$F_{n-1} \geq 2^{(n-1)/2}$$

and:

$$F_{n-2} \geq 2^{(n-2)/2}$$

Because Fibonacci uses two previous values, this proof effectively uses **strong induction**.

2.11 Inductive Step

We know:

$$F_n = F_{n-1} + F_{n-2}$$

Using the induction assumptions:

$$F_n \geq 2^{(n-1)/2} + 2^{(n-2)/2}$$

Since:

$$2^{(n-1)/2} \geq 2^{(n-2)/2}$$

we have:

$$F_n \geq 2(2^{(n-2)/2})$$

Using:

$$2 = 2^1$$

we get:

$$F_n \geq 2^{1+(n-2)/2}$$

and:

Therefore, the result has been proved.

$$F_n \geq 2^{n/2}$$

$$1 + \frac{n-2}{2} = \frac{n}{2}$$

2.12 Closed-Form Formula for Fibonacci Numbers

The direct formula: This formula is commonly known as **Binet's Formula**.

$$F_n = \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right]$$

2.13 The Golden Ratio

Define:

$$\phi = \frac{1+\sqrt{5}}{2}$$

Approximately:

$$\phi \approx 1.6180339887$$

This is called the **golden ratio**.

The second quantity is:

$$\psi = \frac{1-\sqrt{5}}{2}$$

Approximately:

$$\psi \approx -0.618$$

So Binet's formula becomes: $\phi_n - \psi_n$

Since:

$$|\psi| < 1$$

the value:

$$\psi n$$

becomes extremely small as n becomes large.

Therefore approximately: $F_n \approx \frac{\phi^n}{\sqrt{5}}$

This explains why Fibonacci numbers grow exponentially.

2.14 Large Fibonacci Numbers

The notes give examples, including demonstrating that Fibonacci numbers become very large very quickly.

$F_{20} = 6765$

$F_{50} = 12586269025$

$F_{100} = 354224848179261915075$

Chapter 3: Computing Fibonacci Numbers

The computational problem is:

Input: An integer:

$$n \geq 0$$

Output

$$F_n$$

This input/output formulation is explicitly used in lectures.

3.1 Input and Output

An **input** is the information supplied to an algorithm. An **output** is the answer the algorithm produces.

For example:

```
</>

Input:
n = 7

Output:
13
```

because:

$$F_7 = 13$$

3.2 First Solution: Recursive Fibonacci Algorithm

```
</>

FibRecurs(n)

if n ≤ 1:
    return n
else:
    return FibRecurs(n - 1) + FibRecurs(n - 2)
```

This algorithm follows the mathematical definition directly.

3.3 Meaning of Recursion

Recursion occurs when a function calls itself.

For example:

```
</>

FibRecurs(5)
```

calls:

```
</>

FibRecurs(4)
```

and:

```
</>

FibRecurs(3)
```

Those calls then make additional calls.

3.4 Understanding the Base Condition

Consider:

```
</>

if n ≤ 1:
    return n
```

If:

```
</>

n = 0
```

the function returns:

```
</>

0
```

If:

```
</>

n = 1
```

the function returns:

```
</>

1
```

No further recursive call is needed.

3.5 Dry Run of FibRecurs(5)

Let us perform a complete conceptual dry run.

We begin with:

```
FibRecurs(5)
```

Since:

```
5 > 1
```

we calculate:

```
FibRecurs(4) + FibRecurs(3)
```

Now:

```
FibRecurs(4)

= FibRecurs(3) + FibRecurs(2)
```

and:

```
FibRecurs(3)
= FibRecurs(2) + FibRecurs(1)
```

Also:

```
FibRecurs(2)
= FibRecurs(1) + FibRecurs(0)
= 1 + 0
= 1
```

Therefore:

```
FibRecurs(3)
= 1 + 1
= 2
```

Then:

```
FibRecurs(4)
= 2 + 1
= 3
```

Finally:

```
FibRecurs(5)
= FibRecurs(4) + FibRecurs(3)
= 3 + 2
= 5
```

Therefore:

$$F_5 = 5$$

3.6 Recursion Tree

The calls can be represented as:

```
                    Fib(5)
                  /        \
              Fib(4)         Fib(3)
             /      \        /     \
        Fib(3)    Fib(2) Fib(2)  Fib(1)
        /   \      /  \    /  \
    Fib(2) Fib(1) F1  F0  F1  F0
     /  \
    F1  F0
```

Look carefully at this tree.

Fib(3) is calculated more than once.

Fib(2) is calculated several times.

Even Fib(1) and Fib(0) are repeatedly calculated. This **repeated computation** is the main reason why the algorithm is inefficient.

3.7 What Is Running Time?

The **running time** of an algorithm measures how much computational work it performs. It can be defined as the number of lines of code executed by Fib Recurs(n).

$$T(n)$$

This gives us a mathematical way to describe the amount of work.

3.8 Running Time for the Base Case

When:

$$n \leq 1$$

The number of lines executed is $T(n) = 2$ because only a small constant amount of work is performed.

3.9 Running Time for $n \geq 2$

For larger values:

$$T(n) = T(n-1) + T(n-2) + 3$$

The two recursive terms represent: while the constant 3 represents the additional operations performed in the current function invocation.

```
</>

FibRecurs(n - 1)
FibRecurs(n - 2)
```

3.10 Small Dry Run of the Running-Time Recurrence

Given:

$$T(0) = 2$$

and:

$$T(1) = 2$$

then:

$$T(2) = 3 + T(1) + T(0)$$
$$= 3 + 2 + 2$$
$$= 7$$

For $n = 3$:

$$T(3) = 3 + T(2) + T(1)$$
$$= 3 + 7 + 2$$
$$= 12$$

For $n = 4$:

$$T(4) = 3 + 12 + 7$$
$$= 22$$

For $n = 5$:

$$T(5) = 3 + 22 + 12$$
$$= 37$$

Work starts increasing very rapidly.

3.11 Why Is Recursive Fibonacci So Slow?

The recurrence has essentially the same branching pattern as Fibonacci itself.

The notes conclude:

$$T(n) \geq F_n$$

and give:

$$T(100) \approx 1.77 \times 10^{21}$$

operations, which they estimate would require roughly **56000 years at 1 GHZ**

3.12 Understanding the 1 GHz Example

A processor operating at:

$$1 \text{ GHz}$$

has a clock frequency of approximately:

$$10^9$$

cycles per second.

If we roughly imagine one counted operation per cycle, then:

$$\frac{1.77 \times 10^{21}}{10^9} = 1.77 \times 10^{12}$$

seconds.

Converting to years:

$$\frac{1.77 \times 10^{12}}{60 \times 60 \times 24 \times 365} \approx 56,000 \text{ years.}$$

This is only an illustrative model; real processors and instructions are more complicated, but it makes the scale of the problem clear.

3.13 Time Complexity of Recursive Fibonacci

A more standard asymptotic analysis gives approximately:

$$O(\phi^n)$$

where:

$$\phi \approx 1.618$$

It is also common to use the loose upper bound:

$$O(2^n)$$

Both tell us the important thing: The naive recursive Fibonacci algorithm has exponential running time. Exponential algorithms become impractical very quickly.

3.14 Space Complexity of Recursive Fibonacci

Although many recursive calls are created, they are not all stored simultaneously.

The maximum recursion depth is approximately:

$$n$$

Therefore, the recursion stack requires:

$$O(n) \text{ space.}$$

This is useful additional information when analyzing an algorithm because time is not the only resource algorithms consume.

Chapter 4: Efficient Fibonacci Algorithm

The next questions could have been asked:

> Instead of repeatedly recomputing previous Fibonacci numbers, why not compute each value once and keep it?

This produces a dramatically better algorithm.

4.1 Hand Calculation

Humans calculate:

```
</>

0, 1
```

then:

```
</>

0 + 1 = 1
```

so:

```
</>

0, 1, 1
```

Then:

```
</>

1 + 1 = 2
```

giving:

```
</>

0, 1, 1, 2
```

Then:

```
</>

1 + 2 = 3
```

giving:

```
</>

0, 1, 1, 2, 3
```

and so on. The notes use this observation to motivate the efficient algorithm.

4.2 FibList Algorithm

The algorithm in the notes

```
</>

FibList(n)

create an array F[0 ... n]

F[0] ← 0
F[1] ← 1

for i from 2 to n:
    F[i] ← F[i − 1] + F[i − 2]

return F[n]
```

4.3 What Is an Array?

An **array** is a collection of elements stored using indexed positions.

For example:

```
</>

F = [0, 1, 1, 2, 3, 5, 8]
```

The indexes are:

```
Index: 0  1  2  3  4  5  6
Value: 0  1  1  2  3  5  8
```

Therefore:

```
F[0] = 0
F[1] = 1
F[5] = 5
F[6] = 8
```

4.4 Meaning of the Assignment Arrow

The notes use:

$$\leftarrow$$

For example:

```
F[0] ← 0
```

This means:

> Store the value 0 in F[0].

Similarly:

```
F[i] ← F[i − 1] + F[i − 2]
```

means:

> Calculate the sum and store it in F[i].

The assignment symbol should not be confused with mathematical equality.

4.5 Dry Run of FibList(6)

We want:

$$F_6$$

Initially:

```
F[0] = 0
F[1] = 1
```

Iteration *i* =2

```
F[2] = F[1] + F[0]
     = 1 + 0
     = 1
```

Array:

```
[0, 1, 1]
```

Iteration i =3

```
F[3] = F[2] + F[1]
     = 1 + 1
     = 2
```

Array:

```
[0, 1, 1, 2]
```

Iteration i =4

```
F[4] = F[3] + F[2]
     = 2 + 1
     = 3
```

Array:

```
[0, 1, 1, 2, 3]
```

Iteration i =5

```
‹/›

F[5] = F[4] + F[3]
     = 3 + 2
     = 5
```

Array:

```
‹/›

[0, 1, 1, 2, 3, 5]
```

Iteration i =6

```
‹/›

F[6] = F[5] + F[4]
     = 5 + 3
     = 8
```

Final array:

```
‹/›

[0, 1, 1, 2, 3, 5, 8]
```

Return:

```
‹/›

8
```

Therefore: $F_6 = 8$

4.6 Dry-Run Table

i	$F[i-2]$	$F[i-1]$	**Calculation**	$F[i]$
2	0	1	0 + 1	1
3	1	1	1 + 1	2

4	1	2	$1 + 2$	3
5	2	3	$2 + 3$	5
6	3	5	$3 + 5$	8

4.7 Running Time

The notes count the running time as:

$$T(n) = 2n + 2$$

so:

$$T(100) = 202$$

under their line-counting model. Compare this with the recursive algorithm's enormous number of operations. This is an extraordinary improvement.

4.8 Big-O Complexity

Ignoring constants and smaller terms:

$$2n + 2$$

becomes:

$$O(n)$$

Therefore:

```
</>

FibList time complexity = O(n)
```

This is called **linear time**.

4.9 Comparison of the Two Fibonacci Algorithms

Feature	Recursive FibRecurs	Iterative FibList
Main idea	Follow recurrence directly	Compute values once
Repeated calculations	Yes	No
Time complexity	Exponential	$O(n)$

Approx. behavior	Very slow	Very fast
Space	$O(n)$ recursion stack	$O(n)$ array
Suitable for large n	Not	Much better

The notes summarize the key lesson as: The right algorithm makes all the difference.

4.10 Extra Improvement: Fibonacci with Constant Space

The notes store every Fibonacci number in an array. But if we need only F_n, we actually need only the previous two values.

We can write:

```
</>

FibEfficient(n)

if n ≤ 1:
    return n

previous ← 0
current  ← 1

for i from 2 to n:
    next ← previous + current
    previous ← current
    current ← next

return current
```

For example, for $n = 6$:

i	**previous**	**current**	**next**
Start	0	1	—
2	0	1	1
3	1	1	2
4	1	2	3
5	2	3	5
6	3	5	8

This version has time: $O(n)$ but only $O(1)$ extra space. Important: This constant-space version is **additional information**. The Week 1 notes use the array-based FibList algorithm.

Chapter 5: Greatest Common Divisor — GCD

The final major part of the notes introduces the **Greatest Common Divisor**.

For integers a and b, the notes define:

$$\gcd(a,b)$$

as the largest integer d that divides both a and b.

Usually, we take the GCD to be non-negative.

5.1 What Does “Divides” Mean?

The notation:

$$d \mid a$$

is read as: d divides a.

It means that there is some integer k such as:

$$a = d \times k$$

For example:

$$3 \mid 12$$

because:

$$12 = 3 \times 4$$

Similarly:

$$4 \mid 20$$

because:

$$20 = 4 \times 5$$

But:

$$5 \nmid 12$$

because 12 cannot be written as 5 multiplied by an integer.

5.2 Example of GCD

Consider:

and:

$$18$$

Divisors of 12:

```
‹/›

1, 2, 3, 4, 6, 12
```

Divisors of 18:

```
‹/›

1, 2, 3, 6, 9, 18
```

Common divisors:

```
‹/›

1, 2, 3, 6
```

The greatest one is:

```
‹/›

6
```

Therefore:

$$\boxed{\gcd(12,18) = 6}$$

5.3 GCD and Simplifying Fractions

The notes motivate GCD using fraction simplification.

Consider:

$$\frac{18}{24}$$

The GCD is:

$$\gcd(18,24) = 6$$

Divide numerator and denominator by 6:

$$\frac{18/6}{24/6} = \frac{3}{4}$$

Therefore:

$$\boxed{\frac{18}{24} = \frac{3}{4}}$$

The largest common divisor removes all common factors in one step.

5.4 Applications of GCD

The notes specifically mention connections with:

Number Theory

Number theory is the branch of mathematics concerned with integers and their properties.

Topics include:

- divisibility;
- primes;
- factors;
- GCD;
- modular arithmetic.

Cryptography

Cryptography studies methods for protecting information.

GCD computations occur in important cryptographic mathematics, including modular arithmetic and public-key techniques.

The Euclidean algorithm is particularly useful because it calculates GCDs efficiently even when numbers are extremely large.

5.5 Computational GCD Problem

The problem can be written formally as:

Input: two integers

$$a,b \geq 0$$

Output

$$\gcd(a,b)$$

The notes use a large example:

$$\gcd(3918848,1653264)$$

This is deliberately large enough to show why checking divisors one by one is not attractive.

Chapter 6: Naive GCD Algorithm

The first algorithm in the notes is:

```
</>

NaiveGCD(a, b)

best ← 0

for d from 1 to a + b:
    if d | a and d | b:
        best ← d

return best
```

The notes estimate its running time as approximately $a + b$, which becomes impractical for very large values.

6.1 Meaning of best

The variable:

```
</>

best
```

stores the largest common divisor discovered so far.

Initially:

```
</>

best = 0
```

Whenever another common divisor is discovered, it replaces best.

6.2 Dry Run: NaiveGCD(18, 12)

Start:

```
</>

best = 0
```

According to the slide algorithm:

```
</>

d = 1 to 18 + 12
d = 1 to 30
```

Now test each candidate.

d	$d \mid 18$?	$d \mid 12$?	**Update best?**
1	Yes	Yes	1
2	Yes	Yes	2
3	Yes	Yes	3
4	No	Yes	No
5	No	No	No

6	Yes	Yes	6
7 onward	No larger common divisor	—	remains 6

At the end:

```
‹/›

best = 6
```

Return: 6

6.3 Why Is the Naive Algorithm Slow?

Suppose:

$$a$$

contains 20 decimal digits.

It may be approximately:

$$10^{20}$$

A loop that performs roughly 10^{20} iterations is completely impractical.

This reveals an important distinction:

> An algorithm can be correct and still be a bad algorithm.

Correctness answers:

```
‹/›

Does it produce the right answer?
```

Efficiency answers:

```
‹/›

How much work is required?
```

A good algorithm usually needs both.

6.4 Additional Note About the Naive Algorithm

We know that a common divisor cannot exceed:

$$\min(a,b)$$

so, a slightly better naive algorithm could stop there instead of $a + b$. However, this does **not** solve the fundamental problem. For very large numbers, checking every candidate divisor is still extremely slow. What we need is a completely different mathematical idea. That idea is the **Euclidean Algorithm**.

Chapter 7: Euclidean Algorithm

The Euclidean Algorithm is one of the oldest known algorithms. Its power comes from an important mathematical property of GCD.

7.1 Division with Remainder

Suppose we divide integer a by integer b.

We can write:

$$a = bq + r$$

where: q is the **quotient**; r is the **remainder**. The remainder satisfies:

$$0 \leq r < b$$

when $b > 0$.

7.2 Example

Divide:

$$17$$

by:

$$5$$

We obtain:

$$17 = 5(3) + 2$$

Therefore:

```
</>

quotient  = 3
remainder = 2
```

We can write:

$$17 \bmod 5 = 2$$

The symbol:

```
</>

mod
```

means **modulo**, or remainder after division.

7.3 Key Lemma Behind Euclid's Algorithm

The notes state that if a' is the remainder when a is divided by b, then:

$$\gcd(a,b) = \gcd(a',b) = \gcd(b,a')$$

Using the more usual symbol r:

$$\boxed{\gcd(a,b) = \gcd(b,r)}$$

where: $r = a \bmod b$, This is the central idea of the Euclidean Algorithm.

7.4 Why Is Lemma True?

Suppose:

$$a = bq + r$$

Rearranging:

$$r = a - bq$$

Suppose d divides both a and b.

Then:

$$a = dx$$

and:

$$b = dy$$

for some integers x and y. Now:

$$r = a - bq$$

Substitute:

$$r = dx - dyq$$

Factor out d:

$$r = d(x - yq)$$

Therefore, d also divides r.

So, every common divisor of a and b is also a common divisor of b and r. The reverse argument also works. Therefore, they have exactly the same common divisors, which means:This simple mathematical observation produces a dramatically faster algorithm.

$$\boxed{\gcd(a,b) = \gcd(b,r)}$$

7.5 Euclidean Algorithm Pseudocode

The notes are:

```
</>

EuclidGCD(a, b)

if b = 0:
    return a

a' ← remainder when a is divided by b

return EuclidGCD(b, a')
```

A programming-style equivalent is:

```
EuclidGCD(a, b)

if b == 0:
    return a

return EuclidGCD(b, a % b)
```

7.6 Why Does b = 0 Stop the Algorithm?

For any positive integer a:

$$\gcd(a,0) = a$$

Why?

Every positive divisor of a also divides zero because:

$$0 = a \times 0$$

The greatest divisor shared by a and zero is therefore a itself. So, when the second value reaches zero, the first value is the answer.

7.7 Small Dry Run: gcd(18, 12)

Start:

$$\gcd(18,12)$$

Compute:

$$18 \bmod 12 = 6$$

Therefore:

$$\gcd(18,12) = \gcd(12,6)$$

Next:

$$12 \bmod 6 = 0$$

Therefore:

$$\gcd(12,6) = \gcd(6,0)$$

Now the second argument is zero.

Return:

$$\boxed{6}$$

Only two remainder operations were required.

Compare this with checking candidate divisors one by one.

7.8 Full Dry Run from the Notes

The notes use:

$$\gcd(3918848,1653264)$$

Let us perform the complete calculation.

Step 1

$$3918848 = 1653264(2) + 612320$$

Therefore:

$$\gcd(3918848,1653264) = \gcd(1653264,612320)$$

Step 2

$$1653264 = 612320(2) + 428624$$

Therefore:

$$\gcd(1653264,612320) = \gcd(612320,428624)$$

Step 3

$$612320 = 428624(1) + 183696$$

Therefore:

$$\gcd(612320,428624) = \gcd(428624,183696)$$

Step 4

$$428624 = 183696(2) + 61232$$

Therefore:

$$\gcd(428624,183696) = \gcd(183696,61232)$$

Step 5

$$183696 = 61232(3) + 0$$

Therefore:

$$\gcd(183696,61232) = \gcd(61232,0)$$

The second argument is now zero.

Hence:

$$\boxed{\gcd(3918848,1653264) = 61232}$$

This is exactly the progression shown in the lecture notes.

7.9 Dry-Run Table for the Euclidean Algorithm

Step	a	b	**Quotient** q	**Remainder** $a \bmod b$
1	3,918,848	1,653,264	2	612,320
2	1,653,264	612,320	2	428,624
3	612,320	428,624	1	183,696
4	428,624	183,696	2	61,232
5	183,696	61,232	3	0

Once the remainder becomes zero:

```
</>

GCD = 61,232
```

7.10 Why Is Euclid's Algorithm So Fast?

At each step, the numbers become substantially smaller. The notes state that the values are reduced approximately by a factor of two, resulting in roughly logarithmic behavior, and note that even GCDs involving 100-digit numbers require only around hundreds of division steps.

The key point is:

> **We do not inspect every possible divisor.**

Instead, we repeatedly replace the problem with a much smaller but mathematically equivalent problem.

7.11 Understanding the Logarithm

The term: $\log n$ describes growth where repeated multiplication or division reduces a problem very quickly. Consider repeatedly dividing 1024 by 2:

```
1024
512
256
128
64
32
16
8
4
2
1
```

Only 10 divisions are required.

Why?

Because:

$$2^{10} = 1024$$

Therefore:

$$\log_2 1024 = 10$$

This is why logarithmic algorithms are generally extremely efficient.

7.12 Standard Complexity of Euclid's Algorithm

The notes describe the runtime as approximately:

$$\log(ab)$$

in terms of reduction steps.

A commonly used standard bound for the number of Euclidean division steps is:

$$O(\log(\min(a,b)))$$

The two descriptions express the same broad message needed here: The number of Euclidean steps grows very slowly compared with the numerical size of the inputs.

7.13 Interesting Connection Between Fibonacci and GCD

A beautiful connection exists between the two major topics in these notes. The worst-case inputs for the Euclidean Algorithm are closely related to **consecutive Fibonacci numbers**.

For example:

$$\gcd(21,13)$$

gives:

$$21 \bmod 13 = 8$$

$$13 \bmod 8 = 5$$

$$8 \bmod 5 = 3$$

$$5 \bmod 3 =$$

$$2\ 3 \bmod 2$$

$$= 1$$

$$2 \bmod 1 = 0$$

Notice:

```
</>

21,13, 8, 5, 3, 2,  1
```

These are Fibonacci numbers in reverse order. This fact is useful in the formal analysis of the Euclidean Algorithm.

Chapter 8: Comparing Naive and Efficient Algorithms

The Fibonacci and GCD examples teach the same fundamental lesson.

Fibonacci

Naive recursive solution

$$O(\phi n)$$

approximately exponential.

Improved iterative solution

$$O(n)$$

linear.

GCD

Naive approach

Approximately:

$$O(a + b)$$

under the counting model used in the notes.

Euclidean Algorithm

Roughly logarithmic in the numerical input size:

$$O(\log(\min(a,b)))$$

division steps.

8.1 Why Algorithm Design Matters

Suppose we make a computer twice as fast. A very inefficient algorithm may still remain unusable. But changing from an exponential algorithm to a linear algorithm can turn a calculation requiring thousands of years into one that finishes almost immediately. This is why algorithm design can be more important than simply buying faster hardware.

Chapter 9: Important Terms from the Notes

Term	Meaning
Algorithm	Step-by-step procedure for solving a problem
Input	Data supplied to an algorithm

Output	Result produced by an algorithm
Efficiency	Amount of resources used by an algorithm
Running time	Amount of computational work performed
Space complexity	Amount of memory required
Linear scan	Checking items one after another
Optimization	Finding a better/best solution according to some objective
Recursion	A function calling itself
Base case	Condition that stops recursion
Recurrence relation	Formula defining a value using previous values
Fibonacci number	Number defined using the previous two Fibonacci numbers
Lemma	Proven supporting mathematical statement
Theorem	Important mathematical statement that has been proved
Induction	Mathematical proof technique based on base and inductive cases
Array	Indexed collection of values
GCD	Greatest Common Divisor
Divisor	Number that divides another without remainder
$d \mid a$	d divides a
Quotient	Integer result of division
Remainder	Amount left after division
Modulo	Operation that returns the remainder
a % b	Programming notation for remainder of a/b

Euclidean Algorithm	Efficient algorithm for computing GCD
Linear time	Running time proportional to n
Exponential time	Running time growing approximately like c^n
Logarithmic time	Running time growing approximately like $\log n$

Chapter 10: What Students Should Learn from Week 1

After studying these notes, a student should be able to explain the difference between a straightforward programming task and a problem for which algorithm design matters. They should understand that two correct algorithms can perform radically differently.

Students should be able to define the Fibonacci sequence:

$$F_0 = 0, \qquad F_1 = 1, \qquad F_n = F_{n-1} + F_{n-2}$$

and calculate small Fibonacci numbers manually. They should understand why:

```
</>

FibRecurs(n)
```

is inefficient: it repeatedly solves the same subproblems. They should also be able to dry-run:

```
</>

FibList(n)
```

and explain why it requires only linear work. For GCD, students should know:

$$\gcd(a,b)$$

means the greatest number dividing both a and b. They should understand the key Euclidean identity:

$$\gcd(a,b) = \gcd(b, a \bmod b)$$

and should be able to repeatedly apply it until the remainder becomes zero. Finally, students should remember the most important message of the entire lecture:

> A correct algorithm is not necessarily an efficient algorithm.

and:

> Choosing the right algorithm can completely change a problem's feasibility.

The final slide makes the same point: the naive algorithm can be too slow, while finding the right algorithm often depends on discovering an important mathematical property of the problem.